

DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

BIOMEDICAL ROBOTICS 2026/27

Assignment 1

Author:

Antonio Mazzotta - 8973098
Francesco Sisani - 8571674
Matteo Spluga - 9185902

Professor:

Maura Casadio
Federica Sassi

Contents

1 Part 1: EMG Signal Preprocessing and Envelope Extraction **2**
1.1 Data organization . . . 2
1.2 Part 1 solution . . . 2
1.2.1 Preprocessing steps for each muscle . . . 3
1.2.2 Comparison between the reconstructed and MCI registered envelopes . . . 4
1.2.3 Overall considerations . . . 5

2 Part 2: MCI Mapping and Cursor Velocity Computation **6**

3 Part 3: Cursor Trajectory Reconstruction and Error Analysis **6**
3.1 Reaching Analysis Code Overview . . . 6

4 Part 4: Target Reaching Performance Analysis **7**
4.1 Target selection . . . 7
4.2 Performance analysis . . . 8

5 Overview and Objective **8**
5.1 Code Pipeline and Mathematical Formulation . . . 9

Introduction

This first assignment of the course essentially consists in steering a cursor with four different muscles. The group was requested to choose a person who would have worn the MCI to record a reach session. This was composed of a setup and calibration phase and a play part. In conclusion, once we had our data, the main objective was to rebuild the cursor from the raw EMG data and compare them.

More in detail, the assignment can be divided in 3 parts:

- Part 1: Band-passing the raw EMG signal in a range of 30-450Hz, then rectifying it and applying a low-pass filter at 5Hz. This is done to obtain envelopes.
- Part 2: Using the gains and the envelope to create the map that goes from muscle intensity to cursor velocity.
- Part 3: Integrating the cursor velocity into a position and animating it with the targets.
- Part 4: Target reaching performance analysis.
- Part 5: Using our cursor to drive a simulated 2-link robot arm.

MCI

An MCI (Myoelectric Computer Interface) system is a hardware and software architecture used for neuro-rehabilitation purposes. The system acquires electric activity from the muscles (EMG signals, which stands for ElectroMyoGraphic signals) via surface sensors placed on the patient's limbs. The signals are elaborated in real time and projected on a display. The patient uses this visual biofeedback to correct their incorrect muscular activation schemes to help their brain re-label correct movements.

1 Part 1: EMG Signal Preprocessing and Envelope Extraction

1.1 Data organization

In this subsection relative to Part 1 of the assignment, there is a brief explanation of the initial phase of the assignment itself, which consists of three main steps:

- Generation and loading of the file $Group_6.mat$
- Data selection
- Information extraction from the table

After having defined the path to find the session registration data (`ID_6_Session_1`), the `make_group_mat` function collects and organizes the original data acquired from the MCI, generating the `Group_6.mat` file. This file contains the EMG signals, action envelopes, cursor positions, reached targets and calibration information. Then, based on the variable "task" value, the corresponding "center-out" or "random" experiment registrations are selected. Here, the focus was associated to the center-out case. Center-out or random tables are stored in a matrix called "`D`" and the corresponding targets are stored in a matrix simply called "`targets`". Finally, necessary data for the elaborations required for Part 1 are extracted from matrix `D`. In the last part, raw EMG signals (expressed in ADC counts) are converted in microvolts.

1.2 Part 1 solution

Part 1 regards the preprocessing of raw EMG signals, allowing to get the activation envelopes starting from raw EMG signals and to compare the results with the ones produced by the MCI system by terms of correlation and temporal difference. The process follows four steps:

- Band-pass filtering
- Rectificaton
- Low-pass filtering
- Normalization

1.2.1 Preprocessing steps for each muscle

The first initial step allows to transform the raw EMG signals in a measurement of the muscular activation. Initially, a band-pass filter (between 30Hz and 450Hz) is applied to eliminate unwanted low-frequency and high-frequency components. Here, a fourth-order Butterworth filter is projected and the DC offset of each EMG signal is also removed. The filter is applied with the function `emgFilt`. Then, the signals gets rectified with the `emgRect` function, making its values positive, and filtered with a fourth-order Butterworth low-pass filter at 5Hz to obtain a more regular trend. This result is called activation envelope and it represents the intensity of muscular activation in time. It is stored in `envOursUV`. Finally, a normalization of these is carried out with respect to the maximum calibration value for each muscle. With this process, values between 0 and 1 are obtained so that the activations of the four muscles are comparable.

Here the plotted signals are shown for the four muscles:

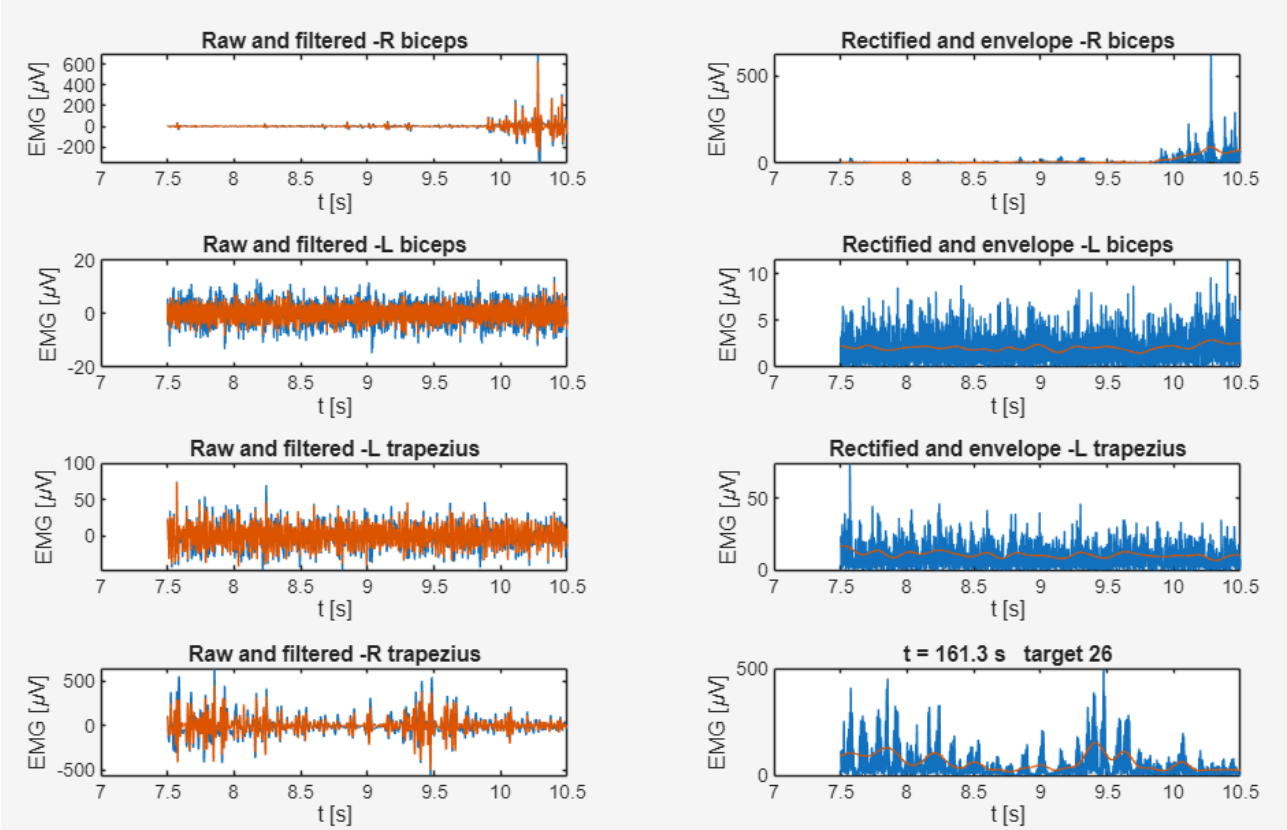


Figure 1: EMG signals processed with the filtering, rectification and DC values elimination processes.
Left images: blue → raw signals, red → filtered signals.
Right images: blue → rectified signals, red → envelopes.

The first set of plots (left column) compares the raw EMG signals with the filtered ones. It can be observed that the band-pass filtering preserves the main oscillations of the EMG signals while removing unwanted frequency components.

The second set of plots (right column) shows the rectified signals and their corresponding envelopes. Rectification makes all signal values positive, while low-pass filtering produces a smoother trend that follows the intensity of muscle activation.

The four muscles exhibit different activation patterns:

- R biceps: shows particularly intense activation towards the end of the selected time window.
- L biceps: exhibits relatively continuous activity distributed over time.
- L trapezius: presents several activation phases, with peaks of significant amplitude.
- R trapezius: shows more intermittent activity, characterized by distinct peaks.

1.2.2 Comparison between the reconstructed and MCI registered envelopes

In the second part, a quantitative comparison is performed between the reconstructed envelopes `envOursNorm` and the ones registered by the MCI system `envMCI`. The correlation matrix between the two envelopes is computed for each muscle to assess how the respective trends are similar. Then, the cross-correlation between signals is computed. With this one, the corresponding delay at the maximum value of the cross-correlation is identified, and it is converted from samples to milliseconds.

Here, the results are shown from the Command Window:

```
Part 1: Correlation matrices
  Muscolo R biceps:
    1.0000    0.8330
    0.8330    1.0000

  Muscolo L biceps:
    1.0000    0.8891
    0.8891    1.0000

  Muscolo L trapezius:
    1.0000    0.8294
    0.8294    1.0000

  Muscolo R trapezius:
    1.0000    0.8593
    0.8593    1.0000
```

(a) Correlation matrices for each muscle

```
Part 1: Delay values for each muscle

Muscolo R biceps: delay =    -40

Muscolo L biceps: delay =   -38.5000

Muscolo L trapezius: delay =   -40.5000

Muscolo R trapezius: delay =   -40.5000
```

(b) Delay values from cross-correlation for each muscle

From the right image it can be seen that the cross-correlation analysis reveals a consistent temporal delay between our reconstructed envelopes and those computed by the MCI system. The estimated delays range from -38.5 ms to -40.5 ms across the four muscles, indicating that our envelopes systematically lead the MCI envelopes by approximately 40 ms.

The similarity of these values across all muscles suggests that the delay is relatively consistent. This behaviour is consistent with the different filtering approaches: our implementation uses zero-phase filtering (`filtfilt`), whereas the MCI operates in real time and therefore introduces a processing delay.

Subsequently, the two envelopes are overlapped, using a time window of 30 seconds, to allow a visual comparison:

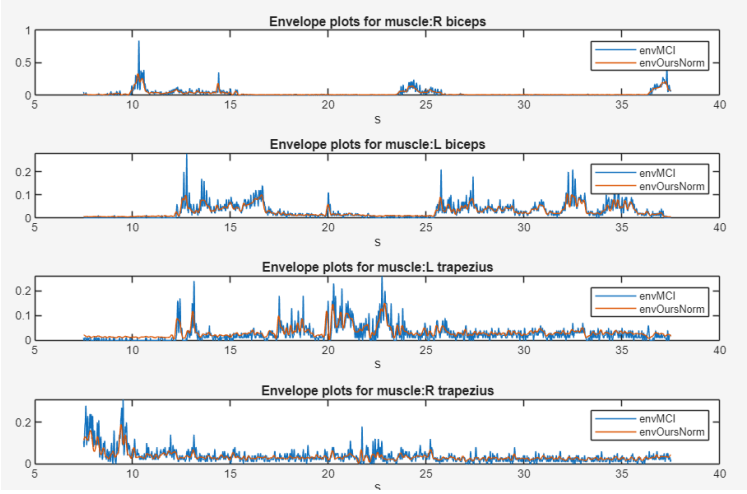


Figure 3: Overlapped MCI and reconstructed envelopes

Here, the main observations are:

- R biceps: the two curves generally follow the same trend, although the MCI envelope exhibits some peaks with higher amplitudes.
- L biceps: a good agreement can be observed between the two envelopes, both during activation phases and periods of rest.
- L trapezius: the curves show an overall similar trend, although some differences in peak amplitudes are present.
- R trapezius: a good overall agreement can be observed, particularly during the initial activation phase, with some differences in subsequent peaks.

At the end, signals are converted from a sampling frequency of 2000Hz to 100Hz. The downsampling is done only after the filtering and the envelopes computation, because raw EMG signals contain high frequency components that would be lost or altered reducing beforehand the sampling frequency.

1.2.3 Overall considerations

The plots show that the implemented preprocessing pipeline produces activation envelopes that generally reproduce the trends of those computed by the MCI system. The most noticeable differences concern the amplitude of some peaks and their temporal alignment.

These discrepancies may be attributed to the different filtering approaches: our implementation uses `filtfilt`, which does not introduce phase delay, whereas the MCI system processes signals in real time.

2 Part 2: MCI Mapping and Cursor Velocity Computation

The second part of the assignment consists of the computation of the cursor velocity starting from the activation envelopes of the four muscles, using the mapping defined by the MCI system. The MCI system uses velocity control, in which the level of muscle activation determines the cursor velocity rather than its position. Each axis is controlled by a pair of muscles: the biceps regulate horizontal movement (X), while the trapezius muscles regulate vertical movement (Y). The velocity along each axis is proportional to the difference between the activation levels of the two associated muscles, so that their simultaneous contraction cancels each other out. The gain, equal to 50 cm/s, determines the maximum cursor velocity when one muscle is activated at 100% and its antagonist is relaxed. These relationships are described by the mapping matrix (W), which allows the velocity components to be calculated from the activation envelopes of the four muscles.
Here the calculations that will determine the speed along each axis is reportedly shown:

```
 Part 2 - MCI map:
x speed = 50 * (R biceps - L biceps)
y speed (down) = 50 * (R trapezius - L trapezius)
```

Then, a dead zone of 5% is applied to the activation envelopes (`envOursD` and `envMCID`). Every activation value below 0.05 is put equal to 0. This procedure allows to eliminate small residual activations caused by noise, avoiding unwanted cursor movements when muscles are at rest.

At last, the cursor velocities are computed using the modified activation envelopes, by applying the MCI map to the two envelopes matrices.

- `velOurs` is the cursor velocity computed using the envelopes reconstructed in Part 1.
- `velMCI` is the cursor velocity computed using the envelopes registered by the MCI system.

3 Part 3: Cursor Trajectory Reconstruction and Error Analysis

This part performs a reaching analysis by simulating the movement of a cursor based on velocity data (Part 2) and comparing it to a desired cursor position. Because the MCI system operates with velocity control, it is necessary to integrate in time the velocities along the two axes to obtain the respective positions. The reconstructed trajectories are then compared with the ones registered during the experiment to evaluate the reconstruction error.

3.1 Reaching Analysis Code Overview

The code initializes position arrays for two methods: one for the custom algorithm (denoted as 'ours') and one for the Minimum Jerk Control (MCI) method. First, the screen boundaries are defined with `screenCm` and the target apparitions by `spawnT`. Then, two trajectories are reconstructed to determine the cursor positions: these are defined by `posOurs`, which was obtained using the activation envelopes from Part 1, and `posMCI`, obtained using the envelopes registered by the MCI system. Both start from the initial position of the cursor registered during the experiment (`cursorCmD`). Then a loop is built to perform three operations:

- Position update, using the relation

$$p(k) = p(k-1) + v(k) \cdot dt \tag{1}$$

  which is based on velocity. This is done at each sample.
- Clipping control, to clip to the screen between [0 0] (top left screen position) and the boundaries defined by `screenCm`, so to ensure that the cursor positions stay within the screen boundaries.
- if resetAtEachTarget is true (reset condition) the positions are reset to the desired cursor position at the current time step. This allows to limit the integration error accumulation at the single phase of target reach.

Finally, after the loop, the code calculates the error between the simulated positions and the desired positions for both methods and prints the mean cursor error for each trajectory (registered and reconstructed) storing them in the `errOurs` and `errMCI` variables. This is done through the function `vecnorm`.

Then, before the animation part, the target coordinates are converted from pixels to cm. This conversion uses the physical dimensions of the screen and its pixel resolution, obtaining the matrix `targCm` which contains the target coordinates expressed in cm. This procedure allows to represent targets and cursor trajectories using the same unit of measurement, facilitating the graphical comparison.

Here an image of the cursor animation relative to the last instant of simulation and the mean cursor error are shown:

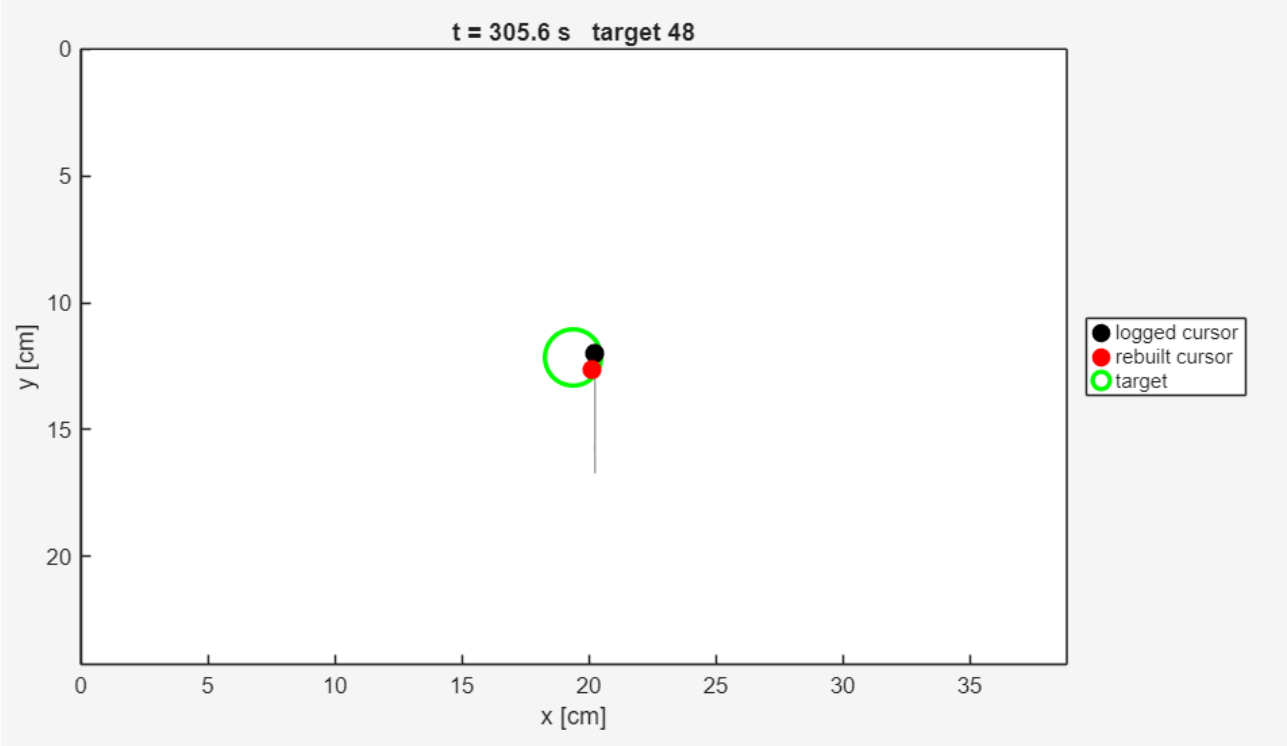


Figure 4: Reconstructed and registered cursor positions, with the relative target, at the simulation last instant of time (t=305.6s, target no.48)

Mean cursor error: ours = 1.942 cm, MCI = 0.486 cm

During the entire animation time the simulation shows that the reconstructed cursor (red) is close to the logged cursor (black), indicating good agreement between the two positions. However, a small discrepancy is still present, highlighting that the reconstructed trajectory does not perfectly reproduce the original one.

By analyzing the mean cursor error, the results show that the reconstruction based on the original MCI envelopes is approximately four times more accurate than the one obtained using our preprocessing pipeline, even though the envelopes produced in Part 1 are similar to the original ones. This difference may be attributed to discrepancies in the envelope extraction process, particularly due to filtering and normalization, as well as the different temporal characteristics of offline and real-time processing. Moreover, `filtfilt` produecs envelopes without phase delay, while the MCI system works in real time and so it has an intrinsic delay.

4 Part 4: Target Reaching Performance Analysis

This part performs a reaching analysis on a set of targets, with the objective to evaluate the effectiveness and the precision of the cursor control during the experiment. The necessary times to reach the targets and the number of trajectory corrections (overshoots) are analysed.

4.1 Target selection

Here, targets that must be included in the analysis are selected. In particular, only the targets that were reached are considered and, in the center-out case, central targets are excluded in order to consider only outwards movements. Instead, in the random task, all the reached targets are considered.

4.2 Performance analysis

Once the target are selected, parameters are extracted:

- Reach time: time spent to reach each target, expressed in seconds.
- Overshoots: number of ”target exceeding” phenomenons or corrections performed during the target reaching phase.

The two parameters are shown here graphically:

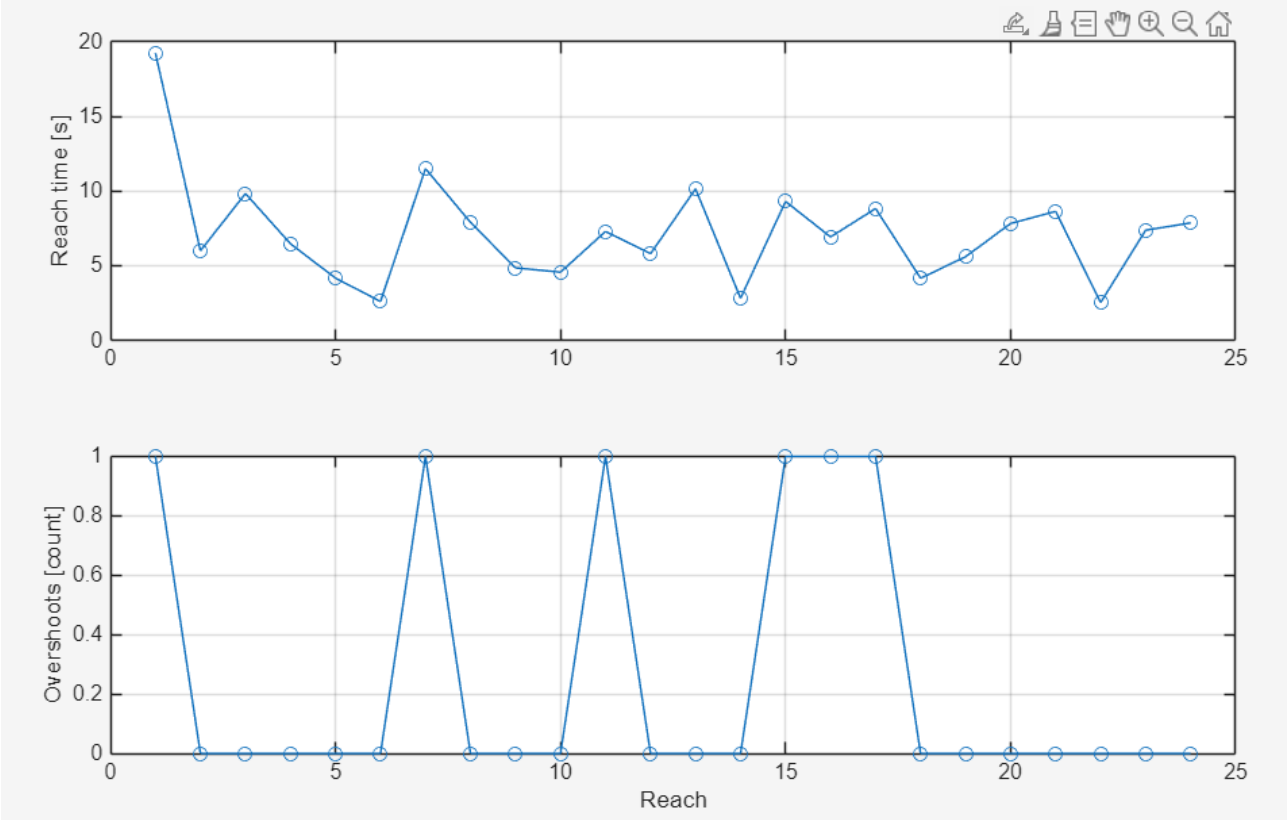


Figure 5: Reach times and overshoot graphics

The upper plot shows the reach time for each of the 24 selected targets. The results exhibit considerable variability, with values ranging approximately from 2.5 to 19 seconds. The first reach presents the longest completion time, while subsequent reaches generally require less time, although no consistent decreasing trend can be observed.

The lower plot shows the number of overshoots for each reach. Most targets are reached without overshoots, while only a few reaches present one overshoot. No cases with more than one overshoot are observed.

Overall, the results indicate that reaching times are relatively variable, whereas the number of overshoots remains generally low. Although there is no clear and continuous improvement throughout the session, the results suggest relatively stable cursor control, with most targets being reached without requiring additional corrections.

5 Overview and Objective

This code implements trajectory tracking (a reaching task) for a planar robot arm with two revolute joints ($L_1 = L_2 = 0.30$ m), driven by the position of a screen cursor. The goal is to compute the joint trajectory $q(t) = [q_1(t), q_2(t)]^T$ required to make the end-effector (hand) follow the spatial reference derived from the screen coordinates.

5.1 Code Pipeline and Mathematical Formulation

1. **Coordinate Mapping (Screen Frame $\rightarrow$ Robot Frame):** Cursor and target coordinates in centimeters relative to the screen frame (y-axis pointing downward) are centered at the origin, scaled to an operational screen width of 0.40 m, inverted along the y-axis, and finally translated into the robot frame ($[0, 0.40]$ m relative to the shoulder):

$$p_{\text{robot}} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \cdot \left(\frac{p_{\text{screen}} - p_{\text{center,cm}}}{100} \cdot S \right) + p_{\text{center,robot}}$$

2. **Reference Velocity Computation:** The desired end-effector velocity $v_{\text{ref}} = [\dot{x}_{\text{ref}}, \dot{y}_{\text{ref}}]^T$ is calculated via numerical differentiation of the reference position over time using MATLAB's `gradient(handRef, dt)` function.

3. **Inverse Kinematics for Initial Condition:** At the first sample ($k = 1$), the initial joint configuration $q(1) = [q_1, q_2]^T$ is solved using analytical inverse kinematics (elbow-down configuration):

$$q_2 = \arccos\left(\frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1 L_2} \right)$$
$$q_1 = \text{atan2}(y, x) - \text{atan2}(L_2 \sin q_2, L_1 + L_2 \cos q_2)$$

4. **Kinematic Integration and Feedback Correction (Resolved-Rate Control):** For each time step k, forward kinematics and the Jacobian matrix $J(q)$ are computed:

$$J(q) = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} & -L_2 s_{12} \\ L_1 c_1 + L_2 c_{12} & L_2 c_{12} \end{bmatrix}$$

The joint velocities $\dot{q}$ are derived by inverting the Jacobian matrix while incorporating a proportional correction term ($K_{\text{corr}} = 10\ \text{s}^{-1}$) to suppress numerical drift and tracking offsets:

$$\dot{q}_k = J(q_k)^{-1} \left(v_{\text{ref},k} + K_{\text{corr}} (p_{\text{ref},k} - p_{\text{arm},k}) \right)$$

Joint positions are then updated using Euler integration: $q_{k+1} = q_k + \dot{q}_k \cdot \Delta t$.

5. **Performance Evaluation and Visualization:** The algorithm computes the mean and maximum tracking errors (in mm) and tracks the minimum value of $|\det(J)|$ to monitor proximity to kinematic singularities. Finally, it plots the spatio-temporal joint trajectories and animates the 2-link robotic arm in real time.